TRƯỜNG: **ĐH CT TP.HCM** KHOA: **CÔNG NGHỆ THÔNG TIN** BỘ MÔN: **KHDL** MH: **TH HỌC MÁY**	**BUỔI 6. HỌC TĂNG CƯỜNG (MARKOV DECISION PROCESS- PHƯƠNG PHÁP Q-LEARNING)**	

A. MỤC TIÊU

- Hiểu và cài đặt được các thuật toán markov decision process và phương pháp q-learning
- Code minh họa.
- Đánh giá được hiệu quả của quá trình sử dụng thuật toán học tăng cường thông qua thực nghiệm.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1. **Sử dụng Google Colab để viết chương trình:**

   B1. Truy cập địa chỉ: https://colab.research.google.com/

   B2. Vào file → new notebook



   B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

2. **MDP (Markov Decision Process)**

MDP là một framework trong học máy và trí tuệ nhân tạo để mô tả và giải quyết các bài toán ra quyết định trong các môi trường ngẫu nhiên, nơi một tác nhân (agent) cần hành động sao cho tối ưu hóa một tiêu chí nào đó theo thời gian. MDP đặc biệt quan trọng trong các ứng dụng liên quan đến học tăng cường (Reinforcement Learning).

Thành phần của MDP: Một MDP được định nghĩa bởi các thành phần chính sau đây:

- **Tập hợp các trạng thái (State Space, S)**: Gồm các trạng thái $s \in S$, biểu diễn tình huống hiện tại của môi trường mà tác nhân đang hoạt động.

  Tập hợp các hành động (Action Space, A): Bao gồm các hành động $a \in A$ mà tác nhân có thể thực hiện khi ở trong một trạng thái nhất định s.

  Hàm xác suất chuyển trạng thái (Transition Probability, $P(s'|s, a)$): Xác suất chuyển từ trạng thái hiện tại s sang trạng thái tiếp theo s' khi thực hiện hành động a. Đây là yếu tố ngẫu nhiên trong MDP.

  Hàm thưởng (Reward Function, R(s, a): Mỗi hành động có thể dẫn đến phần thưởng ngay lập tức R(s, a), biểu thị lợi ích (hoặc chi phí) mà tác nhân nhận được sau khi thực hiện hành động a tại trạng thái s.

  Hệ số chiết khấu (Discount Factor, γ):Thường có giá trị trong khoảng $0 \leq \gamma \leq 10$. Giá trị này biểu diễn mức độ quan tâm đến phần thưởng tương lai của tác nhân. Giá trị càng gần 1 thì phần thưởng trong tương lai càng được coi trọng.

Chính sách (Policy)

Chính sách (Policy) $\pi(a|s)$ là một hàm chỉ dẫn tác nhân hành động trong mỗi trạng thái. Chính sách có thể là:

- **Chính sách xác định (Deterministic Policy)**: Tác nhân chọn một hành động duy nhất $a=\pi(s)$ trong mỗi trạng thái s.
- **Chính sách ngẫu nhiên (Stochastic Policy)**: Chính sách được biểu diễn dưới dạng xác suất, tức là xác suất chọn hành động a tại trạng thái s: $\pi(a|s)=P(a|s)$.

Hàm giá trị (Value Function): Hàm giá trị được sử dụng để đánh giá mức độ "tốt" của một trạng thái hoặc một hành động trong ngữ cảnh của một chính sách.

Hàm giá trị trạng thái V(s):

- Giá trị của trạng thái s dưới một chính sách π là giá trị kỳ vọng của tổng các phần thưởng mà tác nhân sẽ nhận được từ trạng thái đó khi tuân theo chính sách π.
- Công thức:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) | s_0 = s, \pi\right]$$

Hàm giá trị hành động Q(s, a):

- Giá trị của một hành động a tại một trạng thái s dưới chính sách π, thể hiện giá trị kỳ vọng của tổng phần thưởng mà tác nhân sẽ nhận được khi thực hiện hành động a ở trạng thái s, và sau đó tuân theo chính sách π.
- Công thức:

$$Q^{\pi}(s, a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) | s_0 = s, a_0 = a, \pi\right]$$

Bài toán tối ưu hóa trong MDP: Mục tiêu của một tác nhân trong MDP là tìm ra chính sách $\pi*$ tối ưu, tức là chính sách tối đa hóa giá trị kỳ vọng của tổng phần thưởng mà tác nhân sẽ nhận được. Do đó, ta có:

$$\pi^* = \arg\max_{\pi} V^{\pi}(s) \text{ cho mọi trạng thái } s \in S.$$

Giải thuật giải quyết MDP: Các phương pháp phổ biến để tìm chính sách tối ưu trong MDP bao gồm:

Phương pháp lặp giá trị (Value Iteration):

- Dựa trên phương trình Bellman để cập nhật giá trị trạng thái cho đến khi hội tụ.
- Cập nhật hàm giá trị trạng thái bằng cách sử dụng công thức:

$$V(s) = \max_{a} \sum_{s'} P(s'|s, a) \left(R(s, a) + \gamma V(s')\right)$$

Phương pháp lặp chính sách (Policy Iteration):

- Thực hiện qua hai bước: đánh giá chính sách (Policy Evaluation) và cải thiện chính sách (Policy Improvement) cho đến khi hội tụ.
- Đánh giá chính sách: tính $V^{\pi}(s)$ cho mọi trạng thái sss theo chính sách π.
- Cải thiện chính sách: cập nhật chính sách bằng cách chọn hành động a để tối ưu hóa hàm giá trị V(s).

Ví dụ (cách ứng dụng thuật toán Markov Decision Process (MDP)).

Mô tả: ví dụ minh họa này sẽ mô phỏng một bài toán quản lý hàng tồn kho tại một cửa hàng. Cửa hàng này phải quyết định mỗi ngày về lượng hàng cần đặt để tối ưu hóa doanh thu (giảm thiểu chi phí lưu trữ và đảm bảo đủ hàng để bán).

Bài toán: Quản lý hàng tồn kho

Giả sử một số thông tin cho hệ thống:

- Hàng tồn kho được mô hình hóa theo các trạng thái trong khoảng từ 0 đến 4, tương ứng với số lượng hàng tồn kho hiện có.
- Các hành động bao gồm việc đặt hàng thêm 0 đến 3 đơn vị hàng mỗi ngày.
- Mỗi ngày, lượng hàng bán ra ngẫu nhiên từ 0 đến 4 đơn vị.
- Mục tiêu là tìm chính sách đặt hàng tối ưu để tối đa hóa lợi nhuận, dựa trên việc cân bằng giữa chi phí lưu trữ và doanh thu từ hàng bán ra.

Phân tích thành phần của MDP

1. **Trạng thái (State)**: Số lượng hàng tồn kho hiện có (0 đến 4).
2. **Hành động (Action)**: Số lượng đơn vị hàng sẽ đặt (0 đến 3).
3. **Hàm chuyển trạng thái (Transition)**: Xác suất chuyển trạng thái dựa trên số lượng hàng bán ra.
4. **Hàm thưởng (Reward)**: Lợi nhuận dựa trên hàng bán ra trừ đi chi phí đặt hàng và lưu trữ.

Bài toán này sử dụng **Value Iteration** để tìm chính sách tối ưu. Code minh họa như sau:

```
# Thư viện cần

    import numpy as np

# Thiết lập các tham số

    states = [0, 1, 2, 3, 4]  # Số lượng hàng tồn kho

    actions = [0, 1, 2, 3]    # Số lượng hàng đặt hàng

    gamma = 0.9               # Hệ số chiết khấu

# Xác suất hàng bán ra mỗi ngày từ 0 đến 4
```

```
    demand_prob = [0.1, 0.2, 0.4, 0.2, 0.1]
# Hàm tính phần thưởng khi ở trạng thái s và thực hiện hành động a
    def get_reward(s, a):
        cost_per_unit = 1       # Chi phí đặt hàng cho mỗi đơn vị
        holding_cost = 0.5      # Chi phí lưu trữ mỗi đơn vị hàng tồn kho
        selling_price = 2       # Giá bán mỗi đơn vị hàng
        reward = -a * cost_per_unit - holding_cost * s  # Trừ chi phí đặt hàng + lưu trữ
        for demand in range(len(demand_prob)):
            sold = min(s + a, demand)  # Số lượng hàng bán ra (tối đa là slg có sẵn)
            reward += demand_prob[demand] * (sold * selling_price)
        return reward
# Hàm chuyển trạng thái khi thực hiện hành động a ở trạng thái s
    def transition(s, a):
        next_state_prob = np.zeros(len(states))
        for demand in range(len(demand_prob)):
            new_inventory = max(0, min(s + a - demand, max(states)))
            next_state_prob[new_inventory] += demand_prob[demand]
        return next_state_prob
# Value Iteration
    def value_iteration(states, actions, gamma=0.9, theta=1e-4):
        V = np.zeros(len(states))
        policy = np.zeros(len(states), dtype=int)
```

```
        while True:
            delta = 0
            for s in states:
                v = V[s]
                action_values = []
                for a in actions:
                    next_state_prob = transition(s, a)
                    action_value = get_reward(s, a) + gamma * np.sum(next_state_prob * V)
                    action_values.append(action_value)
                best_action_value = max(action_values)
                best_action = actions[np.argmax(action_values)]
                policy[s] = best_action
                V[s] = best_action_value
                delta = max(delta, abs(v - V[s]))
            if delta < theta:
                break
        return V, policy
# Chạy thuật toán value iteration
    V, policy = value_iteration(states, actions, gamma)
# In kết quả
    print("Giá trị tối ưu của từng trạng thái:")
    for s in states:
```

```
        print(f"Trạng thái {s}: {V[s]:.2f}")


    print("\nChính sách tối ưu:")
    for s in states:
        print(f"Ở trạng thái {s}, đặt hàng {policy[s]} đơn vị.")
```

Kết quả sau khi chạy mã trên:

- Giá trị tối ưu của từng trạng thái: Cho biết giá trị mong đợi của từng trạng thái tồn kho.
- Chính sách tối ưu: Cho biết số lượng hàng cần đặt khi ở từng trạng thái để tối ưu hóa lợi nhuận.

```
Giá trị tối ưu của từng trạng thái:
Trạng thái 0: 2.50
Trạng thái 1: 3.10
Trạng thái 2: 3.55
Trạng thái 3: 4.00
Trạng thái 4: 4.50

Chính sách tối ưu:
Ở trạng thái 0, đặt hàng 3 đơn vị.
Ở trạng thái 1, đặt hàng 2 đơn vị.
Ở trạng thái 2, đặt hàng 1 đơn vị.
Ở trạng thái 3, đặt hàng 1 đơn vị.
Ở trạng thái 4, đặt hàng 0 đơn vị.
```

Kết quả trên phản ánh:

- Khi tồn kho ở trạng thái thấp (ví dụ: trạng thái 0), chính sách tối ưu là đặt hàng nhiều đơn vị để đảm bảo đủ hàng để bán.

- Khi tồn kho ở trạng thái cao (ví dụ: trạng thái 4), chính sách là không đặt thêm hàng để tránh chi phí lưu trữ.

3. Phương pháp Q-Learning

Một thuật toán học tăng cường (Reinforcement Learning) không yêu cầu mô hình (model-free), giúp một tác nhân (agent) học cách thực hiện các hành động tối ưu trong một môi trường để đạt được mục tiêu. Thuật toán này dựa trên việc đánh giá chất lượng (Q) của từng hành động tại các trạng thái để đưa ra quyết định tối ưu, thậm chí khi môi trường không xác định trước.

Các Thành Phần Chính của Q-Learning

- **Trạng thái (State, S)**: Mô tả tình trạng hiện tại của tác nhân. Tập hợp tất cả các trạng thái có thể có gọi là không gian trạng thái.
- **Hành động (Action, A)**: Mỗi hành động mà tác nhân có thể thực hiện tại một trạng thái. Tập hợp tất cả các hành động có thể gọi là không gian hành động.
- **Phần thưởng (Reward, R)**: Giá trị phần thưởng tác nhân nhận được khi chuyển từ trạng thái này sang trạng thái khác bằng một hành động cụ thể. Mục tiêu của Q-Learning là tối đa hóa tổng phần thưởng qua các bước.
- **Hàm giá trị hành động (Q-Value Function, Q(s, a))**: Đại diện cho "chất lượng" của hành động a tại trạng thái s, tức là giá trị dự kiến của việc thực hiện hành động aaa tại trạng thái s và tiếp tục với chính sách tối ưu.

Công Thức Cập Nhật Q-Learning: sử dụng công thức cập nhật hàm Q-value để điều chỉnh dần dần ước tính về giá trị của từng hành động trong các trạng thái:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

- $Q(s,a)$: Q-value của trạng thái s khi chọn hành động a.
- α: Tốc độ học (learning rate), xác định mức độ điều chỉnh Q-value mới.
- r: Phần thưởng tức thời nhận được khi chuyển từ trạng thái s sang trạng thái s'.
- γ: Hệ số chiết khấu (discount factor), xác định tầm quan trọng của phần thưởng trong tương lai so với phần thưởng hiện tại.
- $\max_{a'} Q(s',a')$: Giá trị Q lớn nhất tại trạng thái kế tiếp s', thể hiện dự đoán tốt nhất về phần thưởng trong tương lai.

Quá Trình Học của Q-Learning là một thuật toán **học tập qua trải nghiệm**:

- Tại mỗi bước, tác nhân chọn một hành động dựa trên Q-values hiện tại (ví dụ: chọn ngẫu nhiên hoặc theo chính sách **ε-greedy** để cân bằng giữa tìm kiếm và khai thác).
- Sau khi thực hiện hành động, nhận phần thưởng r và trạng thái mới s′.
- Cập nhật Q(s,a) dựa trên công thức trên.
- Lặp lại quá trình cho đến khi Q-values hội tụ.

Chính sách ε-greedy

giúp tác nhân cân bằng giữa **khai thác** hành động có giá trị Q lớn nhất (tức là hành động tốt nhất theo kiến thức hiện tại) và **thử nghiệm** các hành động khác (khám phá) để tìm giá trị tối ưu.

- Với xác suất ε, tác nhân chọn một hành động ngẫu nhiên.
- Với xác suất 1−ε, tác nhân chọn hành động có Q-value lớn nhất tại trạng thái hiện tại.

Ví dụ sử dụng Q-Learning để giúp một tác nhân tìm đường đi trong một lưới 5x5, từ vị trí bắt đầu đến mục tiêu.

```
# Thêm thư viện

    import numpy as np

    import random

# Kích thước của lưới

    grid_size = 5

# Khởi tạo Q-table với các giá trị bằng 0

    Q = np.zeros((grid_size * grid_size, 4))  # 4 hành động: lên, xuống, trái, phải

# Các thông số Q-Learning

    alpha = 0.1         # Tốc độ học

    gamma = 0.9         # Hệ số chiết khấu

    epsilon = 0.2       # Xác suất khám phá

# Hàm chuyển đổi từ trạng thái (x, y) sang chỉ số trong Q-table
```

```
        def state_to_index(x, y):
            return x * grid_size + y
# Hàm lấy trạng thái mới và phần thưởng khi thực hiện hành động
        def step(x, y, action):
            if action == 0:     # Lên
                x = max(0, x - 1)
            elif action == 1:   # Xuống
                x = min(grid_size - 1, x + 1)
            elif action == 2:   # Trái
                y = max(0, y - 1)
            elif action == 3:   # Phải
                y = min(grid_size - 1, y + 1)
    # Phần thưởng
            if (x, y) == (grid_size - 1, grid_size - 1):
                return x, y, 10   # Đạt mục tiêu, phần thưởng lớn
            else:
                return x, y, -1   # Phần thưởng nhỏ cho mỗi bước
# Thuật toán Q-Learning
        for episode in range(1000):
            x, y = 0, 0  # Vị trí bắt đầu
            while (x, y) != (grid_size - 1, grid_size - 1):
                state = state_to_index(x, y)
```

```
    # Chọn hành động theo ε-greedy
        if random.uniform(0, 1) < epsilon:
                action = random.randint(0, 3)  # Chọn ngẫu nhiên
        else:
                action = np.argmax(Q[state])   # Chọn hành động tốt nhất hiện tại
    # Thực hiện hành động
        new_x, new_y, reward = step(x, y, action)
        new_state = state_to_index(new_x, new_y)
    # Cập nhật Q-value
        Q[state, action] = Q[state, action] + alpha * (reward + gamma *
np.max(Q[new_state]) - Q[state, action])
    # Cập nhật trạng thái hiện tại
        x, y = new_x, new_y
# Kiểm tra chính sách học được
    x, y = 0, 0
    path = [(x, y)]
while (x, y) != (grid_size - 1, grid_size - 1):
            state = state_to_index(x, y)
           action = np.argmax(Q[state])
            x, y, _ = step(x, y, action)
           path.append((x, y))
print("Đường đi tối ưu tìm được từ Q-Learning:", path)
```

II. Bài tập ở lớp

Bài 1. Tối ưu lộ trình cho xe giao hàng

Một công ty giao hàng có một chiếc xe đi qua một tuyến đường với nhiều địa điểm khác nhau. Công ty muốn tối ưu hóa lộ trình để giảm thời gian di chuyển và chi phí nhiên liệu. Trên đường, xe có thể gặp các tình huống giao thông khác nhau (tắc đường, đường thông thoáng, v.v.), ảnh hưởng đến tốc độ và chi phí nhiên liệu.

Thông tin bài toán:

Trạng thái: Vị trí hiện tại của xe trên tuyến đường.

Hành động: Lựa chọn lộ trình từ một điểm đến một điểm khác.

Xác suất chuyển trạng thái: Xác suất tắc đường ảnh hưởng đến việc chuyển từ vị trí này sang vị trí khác.

Phần thưởng: Số tiền tiết kiệm được khi tối ưu hóa lộ trình di chuyển.

Yêu cầu bài toán:

1. Mô hình hóa bài toán dưới dạng MDP.
2. Sử dụng Value Iteration để tìm lộ trình tối ưu.
3. Mã Python mô phỏng các trạng thái và lộ trình của xe dựa trên chính sách tối ưu tìm được.

Bài 2. Quản Lý Hàng Tồn Kho

Một cửa hàng cần quyết định hàng tuần về số lượng hàng cần đặt để tối ưu hóa lợi nhuận và giảm chi phí lưu kho. Nhu cầu mỗi tuần là ngẫu nhiên, và cửa hàng chỉ có sức chứa giới hạn

cho số lượng hàng tồn kho.

Thông tin bài toán:

Các trạng thái: Số lượng hàng tồn kho vào đầu mỗi tuần.

Các hành động: Số lượng hàng sẽ đặt hàng thêm.

Xác suất chuyển trạng thái: Dựa trên lượng hàng bán ra trong tuần.

Phần thưởng: Lợi nhuận từ hàng bán ra trừ chi phí đặt hàng và lưu kho.

Yêu cầu bài toán:

1. Mô hình hóa bài toán này dưới dạng MDP với phần thưởng và xác suất chuyển trạng thái.
2. Sử dụng thuật toán Value Iteration hoặc Policy Iteration để tìm ra chính sách tối ưu về số lượng hàng đặt mỗi tuần.
3. Viết mã Python mô phỏng bài toán.

Bài 3. Tối Ưu Hóa Chiến Lược Quảng Cáo

Một công ty muốn tối ưu hóa chi phí quảng cáo để đạt doanh thu tối đa. Mỗi ngày, công ty có thể chọn các chiến lược quảng cáo: Thấp, Trung bình, Cao. Số lượng khách hàng đạt được phụ thuộc vào chiến lược quảng cáo và thay đổi ngẫu nhiên.

Yêu cầu:

1. Xây dựng mô hình Q-Learning với các trạng thái là lượng khách hàng hiện tại và các hành động là các chiến lược quảng cáo (thấp, trung bình, cao).
2. Sử dụng thuật toán Q-Learning để tối ưu hóa quyết định chọn chiến lược quảng cáo hàng ngày.
3. Viết mã Python mô phỏng Q-Learning cho bài toán này.

Bài 4. Thực hiện lại bài 1 và 2 với Q_Learning.

III. Bài tập về nhà

Bài 1. Điều hướng Robot trong Mê Cung

Một robot đang ở trong một mê cung hình chữ nhật có kích thước 5x5. Robot muốn tìm cách đi từ ô bắt đầu (0, 0) đến ô đích (4, 4), trong khi tránh các ô có vật cản. Mỗi bước đi từ một ô sẽ có xác suất 0.1 là robot trượt sang ô liền kề theo một hướng ngẫu nhiên.

Yêu cầu:

1. Mô hình hóa mê cung như một MDP với các thành phần trạng thái, hành động, xác suất chuyển trạng thái và phần thưởng.
2. Sử dụng thuật toán Value Iteration để tìm ra chính sách di chuyển tối ưu.
3. Thực hiện một đoạn mã Python để mô phỏng việc đi qua mê cung theo chính sách tìm được.

Gợi ý:

Hành động: Các bước di chuyển (trên, dưới, trái, phải).

Phần thưởng: +10 khi đến ô đích, -1 cho mỗi bước đi, và -10 nếu rơi vào ô có vật cản.

Bài 2. Lập Kế Hoạch Sản Xuất Tối Ưu

Một công ty sản xuất muốn lập kế hoạch sản xuất trong 3 tháng tới để tối đa hóa lợi nhuận. Nhu cầu thị trường biến động ngẫu nhiên mỗi tháng, và công ty phải quyết định mỗi tháng có sản xuất bao nhiêu sản phẩm dựa trên mức tồn kho hiện có.

Thông tin bài toán:

Các trạng thái: Số lượng sản phẩm tồn kho ở đầu mỗi tháng.

Các hành động: Số lượng sản phẩm sản xuất trong tháng.

Xác suất chuyển trạng thái: Phụ thuộc vào nhu cầu thị trường mỗi tháng (có thể chọn các mức như cao, trung bình và thấp).

Phần thưởng: Tương ứng với doanh thu bán sản phẩm và chi phí sản xuất, chi phí lưu kho.

Yêu cầu:

1. Mô hình hóa bài toán này bằng MDP.
2. Sử dụng Policy Iteration để tìm chính sách tối ưu về số lượng sản phẩm cần sản xuất mỗi tháng.
3. Viết đoạn mã Python để tính toán và đánh giá chính sách tối ưu.

Bài 3. Thực hiện lại bài 1 và 2 với Q_Learning.